

A Distributed Architecture for Training and Inference of RandomForest Models

*Note: Sub-titles are not captured in Xplore and should not be used

Elisa Marzioli
Department of Information Technology
University of Rome Tor Vergata
Rome, Italy
elisa.marzioli@students.uniroma2.eu

Abstract—This paper presents a fault tolerant distributed architecture for training and inference of RandomForest models, evaluated on two different tasks of classification and regression. A model-parallel approach is implemented through a master-worker system that parallelizes training at the level of individual trees: each worker trains a subset of a single logical forest's trees, each tree drawing its own bootstrap sample from the full, shared training set. Inference redistributes tree indices across live workers and aggregates on the master via soft voting (classification) or averaging (regression), following the same scheme a single-machine RandomForest uses internally. Finally, a performance evaluation and comparison with the non-distributed baseline model is carried on. Following a deployment on AWS to ensure fault-tolerance and computational power, the system reaches 99.05% accuracy on classification (SDSS DR14, distributed) against 99.10% for a non-distributed baseline and 98.95% out-of-bag, and an RMSE of 52,779 on regression (California housing) against 52,664 for the baseline. Results confirm that the distributed architecture implemented is to be considered statistically equivalent to a non-distributed baseline, while yielding improved performance in both training and inference, as a synthetic-data scalability study (10k–50k rows, 1–8 workers) shows sub-linear but substantial speedup: $5.4\times$ – $7.2\times$ at 8 workers depending on dataset size.

Keywords— *distributed machine learning, RandomForest, model parallelism, stateless service, gRPC, AWS, fault tolerance, performance analysis*

I. INTRODUCTION

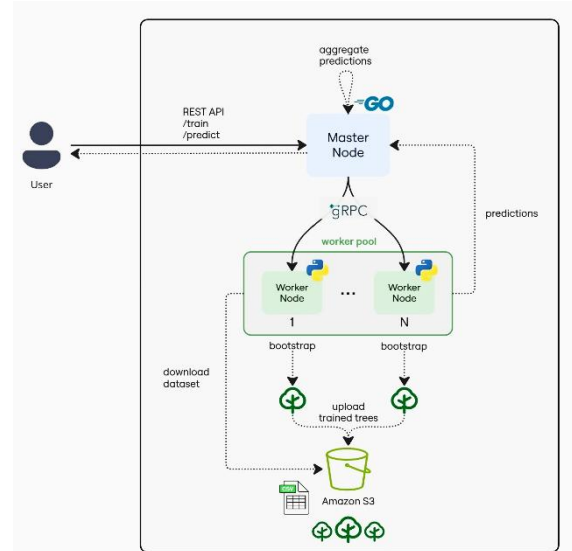
As datasets become increasingly greater in size and complexity, the ability to train Machine Learning models on their data is slowed down by the constraints of a single machine. For this reason, studies that explore distributed training present it as a solution that may provide useful benefits in terms of reduced training time at the cost of minimal losses of accuracy, that have nonetheless been tackled in multiple relevant studies in the past decades [3].

RandomForest appears as the perfect fit for distributed learning: as an ensemble of independent, uncorrelated trees trained via a bagging technique, which by its nature requires its workload to be split across nodes with little coordination, and partial results to be aggregated after the fact without need to synchronize during training. The bagging technique requires in fact a first “bootstrap” phase, where for each tree a so called *bootstrapped dataset* — a dataset randomly sampled from the original dataset, meant to represent a unique sample of the true distribution — is used to train the tree, followed by an aggregation phase where results from the uncorrelated trees are aggregated into a final prediction, resulting in a variance reduction for the final model.

Because every tree — distributed or not — draws its bootstrap sample from the same full training set, the distributed model is expected to be a statistically equivalent estimator to the non-distributed baseline given the same total tree count (§VII). Compared to the baseline, we expect reduced training and inference time as data or tree count grows, at the cost of coordination overhead (network round-trips, per-tree scheduling, result aggregation) which may be more visible in datasets of smaller size.

II. ARCHITECTURE

A. The system



A user contacts a master node to query RandomForest models, requesting training of a certain-sized model using specific parameters, or inference on previously trained models providing a set of features. The master orchestrates a pool of workers, responsible for the distributed training and inference using subsets of trees of the forest. The master is also responsible for aggregating the partial results coming from the worker, in order to send an appropriate response to the client.

- **Training:** the master persists request metadata to a shared storage, assigns trees to workers by distributing indices, and dispatches per-tree training in parallel. Each tree trains on a bootstrap sample of the full training set, extracted by the worker, and uploads to the shared storage as soon as it's done.
- **Inference:** upon receiving a request to use a specific model, the master redistributes analyses the trained trees available for that model on the shared storage and splits the tree indices across the currently healthy workers, collects one prediction per tree, and

aggregates via soft voting (for classification) or averaging (for regression). Aggregation is performed only once, in both cases.

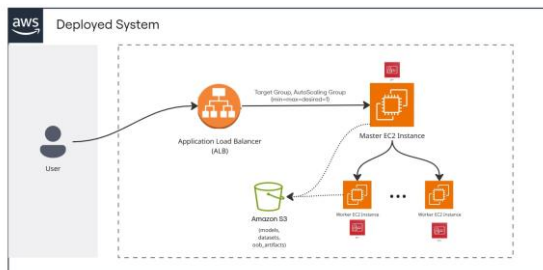
B. Pattern

- Master-worker, centralized orchestration: the master decides partitioning, participation, and aggregation; workers don't carry any orchestration logic.
- Polyglot Microservices: master and workers are implemented in different programming languages, each chosen according to its functions; communication despite independence of the services is guaranteed thanks to gRPC.
- Scatter-gather: both flows scatter to every healthy worker in parallel and gather results for aggregation.
- Stateless workers: no state survives between gRPC calls; all durable state (metadata, trees) lives on shared storage. Because of this, a worker that trained a tree is not necessarily the one that later serves it, and a worker crash loses only in-flight work.
- Server-side discovery (client to master): use of a simple load balancer.
- Cold standby for the master: an autoscaler (AWS ASG) is responsible for keeping the master alive. Reconciliation logic was implemented such that, on boot, a new master whatever a previous master left unfinished.

III. IMPLEMENTATION

A. Infrastructure

The master exposes a REST API to clients and talks to workers over gRPC, using a shared Protocol Buffers contract. The gRPC protocol was chosen for its support to polyglot services, which allowed us to program the master in GO, as it is responsible for orchestration of parallel tasks easily implemented with goroutines, and the workers in Python, for the extensive support of ML libraries needed for the machine learning tasks. Workers use scikit-learn for tree training/inference. Both read and write a shared S3-compatible store: Amazon S3 on AWS, MinIO locally via Docker Compose. The AWS deployment adds EC2 to deploy instances, an Application Load Balancer and Target Group in front of the master, and an Auto Scaling Group holding exactly one master instance alive at all times to guarantee fault tolerance.



B. API and contracts

The REST API (Gin, internal/api) exposes the following methods:

- POST /train: (dataset_url, task_type, target_column, n_estimators, optional hyperparameters). Asynchronous call, returns 202 with a fresh model_id and status="training" immediately; training runs asynchronously from the request context, so neither client disconnect nor a later master crash affects an already-started job (recovery is reconciliation, not context propagation).
- GET /models/{model_id} Reports model status (not_found / training / ready / failed) by inspecting S3.
- POST /predict/{model_id} (synchronous); returns once every live worker has answered (or failed) and results are aggregated.
- GET /health: Used by the ALB Target Group.

Regarding gRPC (api/proto/worker/v1/worker.proto), service Worker exposes: Train, Predict, Health. TrainRequest carries the full dataset URL, task type, target column, a hyperparameters map, and an explicit list of tree indices assigned to that worker. PredictRequest carries the same kind of explicit index list, letting the master reassign a worker's trees to a replacement mid-request.

Hyperparameters are sent from the master to the worker through a generic map. The master (internal/api/hyperparameters.go) validates every key against a configured allow-list (max_depth, max_features, min_samples_split, min_samples_leaf, criterion, max_leaf_nodes, class_weight) and checks each value's shape.

C. Fault tolerance

During training, each worker uploads a tree to S3 as soon as it is trained, rather than waiting for its full batch. If a worker crashes, only the tree in progress is lost; the master detects the missing index and reassigns it to another worker, up to a limited number of retries. If the master itself crashes, a replacement instance checks S3 on startup and resumes training only the missing trees under the original model ID. This is possible because each tree's training data is derived deterministically from the model ID and tree index, so a resumed tree is identical to the one that would have been produced without the interruption.

During inference, workers hold no state, so any worker can compute the prediction for any tree. If a worker fails while serving a request, its trees are reassigned within the same request. If some trees remain unreachable after retries, the result is still returned based on the trees that responded, with the missing indices reported; only a complete failure of all trees returns an error.

The master's own availability is handled at the infrastructure level: it runs as a single EC2 instance behind a Load Balancer, with an Auto Scaling Group configured to keep exactly one instance running at all times. A failed instance is replaced automatically and registered once it passes its health check. This mechanism is only effective in combination with the S3-

based recovery logic described above, without which a new master instance would have no record of in-progress work.

D. Machine Learning Model

The study evaluates the system on two tasks: classification on SDSS DR14 (Sloan Digital Sky Survey, ~10,000 rows, 3-class STAR/GALAXY/QSO) [4] and regression on California housing (median_house_value from location, room counts, and income) [5]. A third, synthetic classification dataset (make_classification, column layout mirroring SDSS) is used for the scalability study (§VI.B), where dataset size must be controlled precisely.

For each tree assigned to it by the master, a worker draws its own bootstrap sample, fits one DecisionTree, and uploads it immediately. This keeps every tree's statistical inputs identical to a single-machine RandomForest's, at the cost of every worker needing the full dataset.

Bootstrap sampling uses a deterministic seed, sha256(model_id:tree_index) truncated to 32 bits (in scripts/bootstrap.py), for both the row draw and the tree's random_state. This lets any worker recompute the exact same sample without exchanging indices, and makes a retried/recovered tree an exact replacement rather than a statistically similar one.

A simple generic preprocessing is performed by the worker on each dataset, expecting the data to be already mostly preprocessed by the user.

To obtain a test set, a standalone script (scripts/split.py) shuffles and splits a source dataset once (fixed seed, optional stratification, using an 80/20 proportion for train and test), uploading <name>_train.csv/<name>_test.csv on the shared storage. This keeps row order canonical for bootstrap sampling and guarantees the baseline sees identical data.

Accuracy (scripts/evaluation.py) is measured by a user run script by calling the real POST /predict once per held-out row compared against the baseline's in-memory scikit-learn predictions on the same rows. Classification reports accuracy; regression reports MSE/RMSE.

To enable validation of the RandomForest model, it's possible to compute the OOB Error of each forest and get an estimate of its generalization error without using a validation set (scripts/evaluate_oob.py): because each tree's bootstrap seed is public and deterministic, a worker recomputes which rows were left out of its sample right after training, tests them, and uploads to the shared storage a small file containing predictions made by the tree on data points that don't belong to its bootstrapped dataset. A separate aggregation step on the master combines every tree's contribution, mirroring RandomForest(oob_score=True) without needing scikit-learn or the seed itself at aggregation time.

IV. TESTING

Brief description of the test suite and the main test case scenarios: fault tolerance (except master failure which is manually tested on aws), REST api, logic

V. EXPERIMENTAL METHODS

All numbers from the AWS campaign (2026-09-18), 100 trees for both tasks; timing over 100 repeated train/predict cycles.

VI. RESULTS

A. Accuracy

Display achieved model accuracy and OOB error or whatever ML metric

B. Training and Inference times

Just put a table with mean and stddev

TABLE I. TABLE TYPE STYLES

Table Head	Table Column Head		
	Table column subhead	Subhead	Subhead
copy	More table copy ^a		

^a Sample of a Table footnote. (Table footnote)

Fig. 1. Example of a figure caption. (figure caption)

Figure Labels: Use 8 point Times New Roman for Figure labels.

VII. PERFORMANCE ANALYSIS

A. Comparison with non-distributed baseline model

Brief description of results of comparison (overhead, accuracy, times) + plots

- are the two models (distributed and baseline) statistically equivalent?

- when to choose one or the other

The baseline (scripts/baseline.py) is a plain, single-threaded RandomForestClassifier/Regressor trained on the same split files, preprocessing, and per-task defaults as the distributed run, driven by the same request. Timing (scripts/benchmark.py) measures what a user would wait for: for the distributed system, training time runs from POST /train to the first ready status (polling included), and inference is one POST /predict; for the baseline, training covers download-through-fit from the same S3 source, inference is one in-memory call. The comparison is asymmetric by construction—the distributed system additionally pays for S3 uploads at training time and downloads at every inference, and that overhead is exactly what the comparison is meant to expose.

The two tasks land on opposite sides of the training comparison, tracking dataset size rather than task type: housing's larger training set (~16,500 vs. SDSS's ~8,000 rows) costs the single-threaded baseline more per tree, while the distributed system's 8 workers absorb that growth by splitting the same 100 trees; SDSS's smaller forest finishes

before the fixed per-request overhead stops being worth paying. This is direct evidence that whether the distributed system is worth it depends on per-tree work relative to coordination overhead, not on task type. Prediction favors the baseline by ~ 2 orders of magnitude on both tasks — the structural cost of the stateless-worker design (§II.B), which doesn't shrink with dataset size.

B. Scalability Analysis

How performance changes as the number of workers increases + results with synthetic data!!!

Three synthetic datasets (10k/20k/50k rows) crossed with worker counts 1/2/4/8 (12 configurations, $n_{\text{estimators}}$ fixed at 100), each repeated 10 times, reporting mean $\pm$ 95% CI.

Parallel efficiency at 8 workers is 0.29–0.54 locally vs. 0.68–0.90 on AWS. Both runs are sub-linear at every size, consistent with a fixed per-worker overhead (gRPC round trip, dataset download, per-tree upload) that doesn't shrink as workers are added; speedup improves with dataset size in both, consistent with that overhead mattering proportionally less as per-tree work grows. The gap between environments traces to contention: on a laptop, 8 worker containers share one CPU and network stack with each other and the master; on AWS, each worker is an isolated EC2 instance. This wasn't specifically tuned for — it's what the same unmodified benchmark measures on two pieces of infrastructure, and it underlines that parallel efficiency depends on the execution environment's ability to run workers independently, not only on the software architecture.

VIII. THREATS TO VALIDITY

Assumptions and comments (eg. performance comparison is not exactly statistically valid as the two models are not statistically equivalent, yet it gives a good understanding of the difference)

Environment threat: AWS Academy Learner Lab instances are shared, budget-constrained hardware, not dedicated — subject in principle to CPU credit throttling or neighbor contention.

IX. LIMITATIONS

Not sure about how this is different from threats to validity

- Retry doesn't distinguish transient failures from deterministic ones (e.g. a malformed dataset) — every failure retries the same way, up to the same budget, wasting retries an error can never recover from.
- OOB error and accuracy are only available via external scripts run after training, not as a live system feature.
- Categorical features are dropped rather than encoded (no slot in the flat-float gRPC contract), with a measurable cost on housing RMSE.
- Evaluating accuracy via one /predict call per row is slow at scale; the validation curve works around this with a 300-row sample per sweep point, trading precision for practicality.
- A training whose metadata was never written to S3 (crash in the narrow window before that first write)

can't be recovered automatically — the user must resubmit.

- No garbage collection for old/abandoned model artifacts on S3.
- No automated test suite for the Python worker code or a full docker-compose end-to-end run against the current code.
- Master→worker topology is a static, manually maintained address list (§II.B), not a dynamic registry.

X. FUTURE WORK

- increase fault tolerance:

- eg. persist user request so that if master fails before starting task the user doesn't need to send it again

- if crash during partitioning there is currently no resume (partitioning starts again from zero), could be interesting for enormous datasets to consider some sort of checkpointing or deterministic shuffling that can be resumed in case of failure

- checkpointing during training to not lose the progress in case of a worker crash

- testing on larger datasets

- include garbage collection logic to clean after old or incomplete models

- Persist the user's request before any work starts, not just before dispatch, so a crash in the earliest window doesn't require resubmission.
- Distinguish transient from deterministic worker failures instead of retrying both identically.
- Expose accuracy/OOB error as a live endpoint rather than an external script.
- Encode categorical features (e.g. one-hot), which requires extending the predict contract beyond a flat float array — would materially improve housing RMSE.
- Run the validation curve on AWS, without the 300-row sampling workaround, if time allows.
- Speed up accuracy evaluation at scale (concurrent /predict calls, or sampling).
- Test on larger, more realistic datasets.
- Garbage-collect old/incomplete models on S3.
- A quantitative comparison against the data-parallelism architecture — both designs are fully implemented, only the comparison itself remains.

XI. CONCLUSION

(tradeoffs, performance, comparison)

This paper presented a master-worker system distributing RandomForest training and inference at individual-tree granularity, and showed — both by construction and by measurement — that this reconstructs the same statistical estimator a single-machine RandomForest would produce: held-out accuracy, baseline accuracy, and OOB error agreed within 0.15 points on classification and 0.2% on regression. The scalability study showed sub-linear but substantial speedup (5.4x–7.2x at 8 workers) and, notably, markedly higher parallel efficiency on

dedicated AWS hardware (0.68–0.90) than on a laptop (0.29–0.54) — a reminder that parallel efficiency is as much a property of the deployment environment as of the software architecture. The baseline comparison showed both outcomes are possible depending on workload: the baseline trained faster on the smaller SDSS forest, the distributed system faster on the larger housing one, which goes on to demonstrate how such distributed architectures provide a growing advantage for larger datasets.

REFERENCES

- [1] L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. <https://doi.org/10.1023/A:1010711814742>.
- [2] Guillaume-Bert, M., & Teytaud, O. (2018). Exact distributed training: Random forest with billions of examples. *arXiv preprint arXiv:1804.06755*.
- [3] Chan, P. K., & Stolfo, S. J. (1993, December). Experiments on multistrategy learning by meta-learning. In *Proceedings of the second international conference on information and knowledge management* (pp. 314-323).
- [4] *Extracted dataset from Sloan Digital Sky Survey*. <https://www.kaggle.com/datasets/lucidlenn/sloan-digital-sky-survey>
- [5] *California Housing prices*. <https://www.kaggle.com/datasets/camnugent/california-housing-prices>
- [6]

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord “Format” pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.